

COMPATIBILITY DOSSIER

Lakehouse Studio

Compatibility-validated installer & operator for open data lakehouses

Master Documentation & Enterprise Plan

v0.6.2 · 17 July 2026
Prepared for product & engineering review

Table of contents

1. Executive summary
2. What Lakehouse Studio is
3. Why it matters — the problem it solves
4. Who it is for — audiences & use cases
5. How it works — architecture & the install pipeline
6. Full capability catalog
 - 6.1 Frontend / operator UX
 - 6.2 Command-line interface (lks)
 - 6.3 Backend services & modules
 - 6.4 HTTP & WebSocket API surface
 - 6.5 The compatibility engine & certification contract
 - 6.6 Stacks & component catalog
 - 6.7 Day-2 operations
 - 6.8 AI-assisted provisioning & troubleshooting
 - 6.9 Companion tools
7. Security posture (current) & the hardening already done
8. Current state — verified facts
9. Council assessment — scored review
10. The road to enterprise grade — step by step
11. Appendix — dependencies, launch, glossary

1. Executive summary

Lakehouse Studio is a UI-driven, compatibility-validated installer and day-2 operator for open data lakehouses. It turns the notoriously painful job of assembling a working open-source lakehouse — picking mutually-compatible versions of a table format, a catalog, one or more query engines, object storage, orchestration and observability, then wiring and operating them — into a guided, "shop for a certified stack" experience.

The product spans the full lifecycle:

1. **Shop** — browse a catalog of components and certified stacks; a live compatibility score guides the cart.
2. **Inspect** — pre-flight checks validate the target host (Docker, RAM, CPU, disk, ports).
3. **Install** — a hardened pipeline (clone → env → doctor → start → bootstrap → smoke) runs with progress streamed live over WebSocket.
4. **Operate** — day-2 tooling: RBAC, audit logging, scheduled backups + DR drills, multi-source ingest, downstream BI destinations, a read-only SQL editor, and an AI assistant grounded in the compatibility matrix.

Its distinctive idea — the **moat** — is that compatibility is treated as a **governed, evidence-backed contract**: every certified stack ships a lock file pinning exact image tags, with a candidate → **pilot-stable** certification lifecycle that only advances on a recorded, passing end-to-end install.

Where it stands today (verified this session): synced to a stable v0.6.2; the automated test suite is **498 passed / 0 failed** (499 on Linux); dependencies install cleanly; and a confirmed prompt-injection→shell RCE in the AI provisioner has been gated. Then, on 2026-07-17, **all 12 stacks were installed end-to-end on a Linux VPS (Finalert srv1541349) and promoted to pilot-stable, each with a recorded evidence[] record** — the four that had never worked (hudi, delta, iceberg-polaris, fintech-compliance) were root-caused and fixed. A five-model council scored the product at **~6.1 / 10 overall** before that campaign, gated at the time by breadth-of-evidence and security hardening; the evidence gap is now closed (12/12 proven), which materially lifts the functionality/evidence concern — **but the security & enterprise-readiness gaps below still gate production use**.

Important — what pilot-stable does and does NOT mean. On the project's own ladder (candidate → pilot-stable → linux-stable → production), pilot-stable means *"installs cleanly and the lakehouse is usable,"* verified with evidence. It is **not** "production-hardened for any organization." Every stack ships with **demo/pilot credentials** (e.g. root/s3cr3t, *_pilot DB passwords) that must be rotated per deployment; there is **no SSO/OIDC** (RBAC is opt-in flags), stacks run on the **host Docker daemon with no sandbox**, images are **tag-pinned and unscanned**, and it is **single-host** (no HA/multi-tenancy). **Net: ready today for pilots, evaluations, dev/test, and internal use by a trusted operator on any of the 12 stacks; for customer-facing / regulated / multi-tenant production, complete the Phase 0–2 hardening in §10 first.**

Where it needs to go: finish the security/sandbox boundary, earn install evidence for the remaining stacks, digest-pin and scan images, and add SSO/enforced-RBAC/multi-tenancy. Section 10 lays out the step-by-step plan.

2. What Lakehouse Studio is

Lakehouse Studio is a self-hostable web application (a FastAPI backend + a single-page HTML frontend + a thin CLI) that installs and operates **open data lakehouses** on Docker. A "lakehouse" here means the open-source stack that gives you warehouse-style SQL over cheap object storage:

- **a table format** — Apache Iceberg, Apache Hudi, or Delta Lake (ACID tables, time-travel, schema evolution over Parquet on object storage);
- **a catalog** — Iceberg REST, Nessie (git-for-data), Hive Metastore, or Apache Polaris (RBAC + credential vending);
- **one or more query/processing engines** — Apache Spark, Trino, StarRocks;
- **object storage** — MinIO (S3-compatible), or cloud (S3/GCS/ADLS) config;
- optional **orchestration** (Airflow, Dagster), **BI** (Superset), **observability** (Prometheus + Grafana + Loki), and **streaming ingest** (Kafka, Flink, Debezium).

Rather than hand-authoring and debugging Docker Compose files and version matrices, the user assembles a stack in a shopping-cart UI, and Studio installs a **known-good, version-pinned combination** and then helps operate it.

It is **not** a hosted SaaS and not a proprietary engine — it is an orchestration/operations layer over best-of-breed open-source components, deployed on infrastructure the user controls (localhost or a remote host over SSH).

3. Why it matters — the problem it solves

Assembling an open lakehouse is hard for reasons that have nothing to do with the user's actual data problem:

- **Version compatibility is a minefield.** A given Spark line only works with specific Iceberg/Hudi/Delta bundle JARs; Trino connectors are version-sensitive to StarRocks and to catalog versions; Hive Metastore needs a specific backing DB; a wrong pairing yields cryptic `ClassNotFoundException/getAllDatabases/schema` errors *after* a long install. (This project's own git history is a catalogue of exactly these scars.)
- **"Works on my machine" isn't a contract.** Compose templates and Helm charts make **zero** compatibility claims — they encode one person's lucky combination with no evidence it reproduces.
- **Day-2 is an afterthought.** Even when a stack boots, operators still need RBAC, audit, backups/DR, ingest, BI hookups and troubleshooting — usually assembled ad hoc.

Lakehouse Studio's answer:

- **Compatibility as a first-class, evidence-backed contract** — every certified stack has a lock file pinning exact tags, an `incompatible[]` list of known-bad combos, and an `evidence[]` record proving a real end-to-end install passed. Version bumps require fresh evidence.
- **A guided, legible install** — a pre-flight that fails fast on the host, and a live-streamed pipeline so the operator sees exactly what happened, with retry/skip/rollback on failure.
- **Batteries-included day-2** — the operational surface (audit, backups, ingest, BI, AI troubleshooting) is part of the product, not a follow-on project.
- **Honesty by design** — stacks are labelled `candidate` until proven; the UI says "pilot/demo, not production-hardened" and ships a hardening checklist rather than overpromising.

4. Who it is for — audiences & use cases

Audience	Why they use it	Representative stack/template
Evaluators / first-time lakehouse users	Get a working Iceberg lakehouse on a laptop in ~30 minutes without learning the whole tool matrix	<code>udp-local-v0.2</code> (Local Demo)
Data platform teams at SMBs / scale-ups	Stand up a real open lakehouse (git-catalog, scheduled ETL, dual-engine SQL) without a dedicated platform team	Production Lakehouse (Nessie + Trino + StarRocks)

Growth-stage startups / first data hire	Sub-second BI over Iceberg without hiring a data-infra specialist	Startup Analytics (Iceberg + StarRocks + Superset)
AI/ML researchers	Iceberg + Spark + Trino + Jupyter for feature/data exploration	AI/ML Research
Regulated-industry teams (fintech / healthcare)	Compliance-shaped templates with lineage, audit, and access control	Fintech Compliance (OpenLineage); Enterprise Hadoop (Ranger/Hive/HDFS)
Platform/DevOps engineers replicating an on-prem cluster	A single-host Docker replica of a bare-metal Hadoop datalake for dev/test	enterprise-hadoop-v1.0
Streaming/CDC teams	Kafka + Flink change-data-capture into the lakehouse	streaming-local-v1.0

Common thread: anyone who needs a *working, trustworthy* open lakehouse and values a guided path + a compatibility guarantee over hand-rolling infrastructure.

5. How it works — architecture & the install pipeline

Shape: a FastAPI + uvicorn backend (backend/main.py:app) serving a single static SPA (frontend/index.html) at /, OpenAPI docs at /docs, a WebSocket log stream, and a Click-based CLI (lks) that is a thin REST/WS client. Default bind 127.0.0.1:7878; launchers run.sh / run.ps1 create the venv, install deps, and start uvicorn — and warn if bound to a non-loopback address without an auth token.

The certified base compose is deliberately frozen byte-for-byte to preserve the lock-file contract. Anything heavy or destructive (TLS via a Caddy sidecar, a monitoring sidecar, JDBC extras) is delivered as an **additive override-compose file** (docker-compose.<name>.yaml) that Studio writes next to the base and hands the operator the exact docker compose ... up -d command to run — Studio never runs up/down on the operator's own stack for those.

The install pipeline (streamed live, per step, over WebSocket) is: **clone** → **env** → **doctor** → **start** → **bootstrap** → **smoke**. Each step reports status; a failed step can be retried, skipped (if skippable), rolled back, or diagnosed by the AI assistant. The same harness is scriptable (scripts/install_harness.py) to produce paste-ready evidence YAML for certification.

(A deeper module-by-module and endpoint-by-endpoint breakdown follows in §6.3–6.4.)

6. Full capability catalog

6.1 Frontend / operator UX

A single-file SPA (frontend/index.html, ~6,300 lines of markup + inline CSS/JS) with 22 component logos. Navigation: **Home** · **Quick Install** · **Installs** · **Settings**, plus a sticky **8-step stepper** (Goal → Build → Name → Size → Connect → Pre-flight → Install → Ready).

The shopping-cart install flow:

1. **Welcome** — "install a lakehouse in 30 minutes", animated data-flow diagram, entry points to the guided flow and the Stack Builder.
2. **Goal / template** — use-case template cards (Local Demo, Production Lakehouse, Startup Analytics, AI/ML Research, Fintech Compliance, Enterprise On-Prem, Streaming, Healthcare).
3. **Quick Install** — one-click deploy of a template, no cart.
4. **Build (the cart)** — the core screen: a per-category component catalog on the left; a sticky cart on the right showing a **live compatibility score /100**, a verdict, chosen components, "next pick" hints, warnings, and **live stack sizing** (recommended vCPU/RAM/disk from the manifest). Actions: reset, use-recommended, validate & continue.
5. **Name** — name input with a generator and validation.
6. **Sizing** — three tiers + a stack quality/maturity score, custom CPU/RAM/disk override, and **matched VPS plans** (Hetzner/DigitalOcean/Linode/Vultr with approximate \$/mo).
7. **Server details** — localhost or remote host, an **SSH credentials** block for remote installs, and the install directory.
8. **Pre-flight inspection** — validates the host against stack requirements; then **Install** or **AI Build** (AI researches versions → generates configs → installs → verifies).

Install & success UX: an **Installing** screen with a live per-step pipeline panel and a terminal-styled WebSocket log stream (with an AI **diagnose** panel on failure and retry/skip/cancel/rollback controls); a **Ready** screen with service URLs, a refreshable **live-health grid**, connection strings, a **"pilot/demo — not production-hardened" hardening checklist**, and the certification-evidence line.

Other surfaces: an **Installs** history screen; a **Settings** screen (alerts & notifications config); a **Stack Builder modal** (custom lakehouse factory with a live `docker-compose.yml` preview, driven to AI Build); and modals for pre-install, build-custom-image, compatibility detail, and "how it works".

Accessibility (VPAT 2.5, V0.6): WCAG 2.1 **A: Yes, AA: Yes** (AAA/508/EN 301 549 not evaluated). Alt text present, good heading structure, no focus traps (Escape closes all modals), 110 keyboard-operable buttons. Known non-support: the animated aurora background lacks a pause control (WCAG 2.2.2) and there is no skip-link / landmark roles (2.4.1) — both on the remediation list.

6.2 Command-line interface (lks)

A Click-based HTTP/WS client over the backend (default `http://127.0.0.1:7878`; `--token/LHS_TOKEN`; `--output table|json`). Command groups:

- **catalog list · templates list · stacks list / compat <id> / upgrades <id>**
- **install create <stack_id> (--host --install-dir --lake --goal) · install status / logs [-f] / retry / skip / cancel**
- **health <install_id> · backup create/list/restore/delete · tables list / describe**
- **ai ask <install_id> "<question>" · export <install_id> -o bundle.tar.gz**

Exit codes 0/1/2; `install logs -f` streams over WebSocket (needs the `websockets` extra).

6.9 Companion tools

superset_dashboards/ — a small standalone FastAPI tool (separate from the main app) that connects to any running Superset, pulls the full dashboard inventory over the Superset REST API, and **exports it to Excel** (read-only). Serves on `127.0.0.1:8099`; per-dashboard columns include title, best-effort team, owners, refresh frequency, tags/roles, and modified/created metadata.

(Sections 6.3 Backend modules, 6.4 API surface, 6.5 Compatibility engine, 6.6 Stacks catalog, 6.7 Day-2, and 6.8 AI provisioning are detailed below.)

6.3 Backend services & modules

The backend is a single FastAPI app (`backend/main.py:app`, ~115 KB — all routes registered directly on app, no separate routers) orchestrating Docker-Compose lakehouse deployments. Three architectural conventions run through it:

- **runner.py and the base compose are FROZEN.** Almost every later feature is a *pure-additive* module — a sibling override-compose file or an event-bus subscriber — never an edit to the certified install path. This is what preserves the lock-file contract.
- **An in-process event bus (events.py) is the backbone.** The runner publishes `LogEvents`; WebSockets, audit logging, backups and notifications all subscribe. It keeps bounded per-install history with monotonic sequence numbers for WebSocket reconnect/replay.
- **State is a single work/state.json file** (`state.py`, atomic writes, restart reconciliation). A v1.0 multi-tenant SQLite schema exists but is **not yet wired** (scaffold).

Module inventory by subsystem:

Subsystem	Modules	What they do
Core / infra	<code>main.py</code> , <code>config.py</code> , <code>paths.py</code> , <code>models.py</code> , <code>state.py</code> , <code>events.py</code> , <code>redact.py</code> , <code>evidence.py</code>	App composition root; env config; install-dir safety (<code>validate_install_dir</code> refuses system/non-empty dirs); Pydantic wire models; JSON install registry; event bus; secret redaction; host-evidence snapshots
Install pipeline	<code>runner.py</code> (UDPRunner), <code>runner_extra_scripts.py</code> , <code>etl_verify_job.py</code> , <code>inspector.py</code> , <code>uninstall.py</code> , <code>structured_smoke.py</code> , <code>stack_manifest.py</code>	The frozen step pipeline (<code>prepare→clone→env→doctor→start→bootstrap→s</code>); Studio-owned bootstrap/smoke scripts; a PySpark job proving Iceberg+Delta+Hudi round-trip; pre-install host inspection; destructive uninstall; per-check smoke cards; manifest loader
Catalog / composition	<code>catalog.py</code> , <code>component_registry.py</code> (~50 KB), <code>stack_composer.py</code> , <code>stack_compose_fragments.py</code> (~53 KB), <code>custom_stack_runner.py</code> (~63 KB)	Component catalog; the single source of truth for buildable components (<code>image/ports/deps/health</code>); full-compose generation from any selection; per-stack compose fragments; end-to-end custom-stack orchestrator

Compatibility	<code>compatibility.py</code> , <code>compat_check.py</code> , <code>compat_explainer.py</code> (~48 KB), <code>compat_ai.py</code> , <code>version_fetcher.py</code>	Lock loading + drift check + image precheck; fast cart verdict; plain-English explainer; LLM version-set proposals; live version discovery
Scoring / sizing	<code>scorer.py</code> , <code>sizer.py</code> , <code>providers.py</code> , <code>cart.py</code> , <code>lake_namer.py</code> , <code>templates.py</code> , <code>compliance.py</code>	100-point stack score; resource sizing tiers; VPS provider matching; live cart score; name generator; template views; compliance framing
AI	<code>ai_assistant.py</code> (~33 KB), <code>ai_provisioner.py</code> , <code>ai_configurator.py</code> , <code>ai_safety.py</code> , <code>error_explainer.py</code>	Grounded "Ask Studio" (Anthropic <code>claude-haiku-4-5</code>); LiteLLM provisioner/configurator; the AI trust boundary (§7) ; failure classifier
RBAC / audit	<code>rbac_auth.py</code> , <code>bootstrap_rbac.py</code> , <code>audit_log.py</code> , <code>v1/rbac.py</code> , <code>v1/multi_tenant_schema.py</code> , <code>v1/executor_interface.py</code>	Opt-in per-user RBAC (SQLite, sha256 tokens); first-OWNER seeding; opt-in audit tee; v1 scaffolds (roles, multi-tenant schema, executor abstraction — not wired)
TLS / hardening	<code>tls_wizard.py</code> , <code>caddy_tls.py</code> , <code>jdbc_extras.py</code>	Self-signed certs + password rotation; Caddy HTTPS overlay; JDBC-jar side-load overlay
Ingest / sources / destinations	<code>ingest.py</code> (~55 KB), <code>data_sources.py</code> , <code>destinations.py</code> , <code>insyght_connector.py</code> , <code>table_explorer.py</code> , <code>sql_editor.py</code> , <code>demo_query.py</code> , <code>data_quality.py</code>	CSV→Iceberg via Spark; encrypted external sources; outbound BI destinations; Insyght connector; read-only Iceberg catalog client; sandboxed read-only SQL; canned demo queries; table assertions
Day-2 ops	<code>backup.py</code> (~37 KB), <code>monitoring.py</code> , <code>notifications.py</code> , <code>service_logs.py</code> , <code>health.py</code> , <code>upgrade_executor.py</code> , <code>gitops_export.py</code> , <code>gitops_import.py</code>	Metadata/full backups + DR drills; monitoring sidecar; multi-channel notifications; log viewer; health probes; destructive upgrade runner with rollback; GitOps bundle export/import
Overlays (opt-in)	<code>airflow_overlay.py</code> , <code>dagster_overlay.py</code> , <code>superset_overlay.py</code> , <code>observability_overlay.py</code>	Additive <code>docker-compose.<name>.yaml</code> overlays toggled by LHS_*_ENABLED flags

6.4 HTTP & WebSocket API surface

~90+ endpoints, all registered on app and reachable under **both** `/api/...` and `/api/v1/...` (an ASGI `v1AliasMiddleware` rewrites the prefix for HTTP and WebSocket alike). Auth is a shared bearer token

(LHS_AUTH_TOKEN) or per-user RBAC when enabled. Grouped:

- **Auth / catalog / metadata** — GET /api/auth/status, /api/catalog, /api/goals, /api/compat-rules, /api/templates, /api/templates/{id}, /api/compliance/{tag}.
- **Versions / AI compat / image build** — GET /api/versions/{id}, POST .../refresh, POST /api/compat-ai/research, .../clear-cache, POST /api/image-build/research|start, GET /api/image-build/status|stream/{job}.
- **Stack builder (custom) + AI provisioner** — GET /api/stack-builder/catalog|presets, POST .../build-template|resolve|compose-preview|build, GET .../status|stream/{job}, POST /api/ai-provision/start, GET /api/ai-provision/status|stream/{job}.
- **Stacks + compatibility + upgrades** — GET /api/stacks, /api/stacks/{id}, .../sizing, .../score, .../compatibility, POST .../compatibility/precheck, GET .../upgrades, POST .../upgrades/simulate, POST /api/installs/{id}/upgrades/execute, GET .../upgrades/executions, GET /api/upgrades/executions/{id}.
- **Cart / lake names / inspection** — POST /api/cart/validate, GET /api/cart/recommended, GET /api/lake-names/suggest, POST /api/lake-names/validate, POST /api/inspect.
- **Install lifecycle** — GET/POST /api/installs, GET /api/installs/{id}, POST .../cancel, .../steps/retry|skip|rollback, GET .../export, POST .../uninstall|control, GET .../health|diagnose, POST .../smoke-structured, **WS** .../logs, GET .../services/{svc}/logs, **WS** .../services/{svc}/logs/stream.
- **TLS / security** — POST .../tls/generate, GET .../tls/certs, DELETE /api/tls/certs/{id}, POST .../tls/caddy/enable|disable, POST .../security/rotate-password|password-strength.
- **Sidecars** — POST .../jdbc/enable|disable, .../monitoring/enable|disable.
- **Backups / DR** — POST/GET .../backups, POST /api/backups/{id}/restore, DELETE /api/backups/{id}, GET/PUT .../backups/schedule.
- **Data (SQL / demo / ingest / sources / destinations / tables / DQ)** — GET /api/demo-queries, POST .../demo-query|sql|uploads|ingest, GET .../ingest[/]{job}], POST .../ingest/postgres|mysql, POST/GET .../data-sources, POST /api/data-sources/{id}/test, DELETE .../{id}, POST/GET .../destinations, GET/POST/DELETE /api/destinations/{id}/[test|provision|connection], GET .../tables[/]{ns}/{name}], POST/GET .../dq/checks, POST /api/dq/checks/{id}/run, GET .../dq/results.
- **Notifications / audit / AI / RBAC / misc** — GET /api/notifications/config, POST .../test, GET /api/audit, GET /api/ai/status, POST /api/ai/ask (+ an SSE streaming variant), POST/GET/DELETE /api/rbac/users, GET /api/rbac/me, GET / (SPA), GET /healthz (liveness + catalog/compat drift).

6.5 The compatibility engine & certification contract

Four cooperating layers make "the matrix is the moat" real:

- **compatibility.py (enforcement)** — loads every stacks/compatibility/*.lock.yaml, checks catalog↔lock drift at startup (surfaced via /healthz), and runs an **install-time image-availability precheck** (docker manifest inspect) so a stack can't start with a vanished tag.
- **compat_check.py (fast machine verdict, <100 ms, no I/O)** — matches a cart to a certified stack (cart ⊆ stack vocabulary, with a "marriage floor" so a single shared component like minio doesn't trivially match), runs the lock's constraints, and returns a 0-100 readiness score.

- **compat_explainer.py (human verdict)** — returns `will_work` / `wont_work` / `untested` with per-pair evidence, a graduation path for candidate matches, and alternative-cart suggestions one swap away.
- **compat_ai.py (LLM fallback)** — proposes compatible version sets when there is no lock (custom builds / AI provisioner), cached 24 h.

The certification contract is covered in §6.6.

6.6 Stacks & component catalog

13 stack manifests (`stacks/*.yaml`), each with a common schema (`id`, `name`, `mode`, `maturity`, `components` with per-tier resource profiles, requirements, ports, env defaults, commands, outputs, certification). The **authoritative runtime status is the lock file's status**, not the manifest's authored `maturity`.

As of 2026-07-17 every stack is pilot-stable — all 12 were installed end-to-end on the Finalert VPS (`srv1541349`, Linux/Docker 29.5.0/31 GB/8 cores) and carry a recorded `evidence[]` record.

Stack	Lock status	One-liner
<code>udp-local-v0.2</code>	pilot-stable	Reference Iceberg + Spark + StarRocks lakehouse on one host (Iceberg REST catalog, MinIO)
<code>udp-trino-local-v0.1</code>	pilot-stable	Same core, Trino 481 as the query engine
<code>hudi-hms-spark-local-v0.1</code>	pilot-stable *	Hudi upserts on Hive Metastore + MySQL via Spark (fixed 2026-07-17: HMS catalog + hadoop-aws)
<code>techsophy-sdp-hadoop-v1.0</code>	pilot-stable	Remote production bare-metal Hadoop cluster (<code>remote-cluster</code> mode — health-check only)
<code>streaming-local-v1.0</code>	pilot-stable	Kafka → Flink → Iceberg → StarRocks real-time pipeline
<code>enterprise-hadoop-v1.0</code>	pilot-stable	Full Hadoop datalake replica in Docker (HDFS/YARN/Hive/Tez/Ranger/Airflow/observability)
<code>startup-analytics-local-v0.1</code>	pilot-stable	Reference core + Apache Superset self-service BI
<code>ai-ml-research-local-v0.1</code>	pilot-stable	Iceberg feature store + Spark + Trino + JupyterLab (heaviest local: 16-24 GB RAM)
<code>fintech-compliance-local-v0.1</code>	pilot-stable *	Iceberg + Trino + StarRocks + OpenLineage/ Marquez (fixed: env-overridable Marquez host ports)

iceberg-nessie-trino-local-v0.1	pilot-stable	Nessie git-catalog + Trino + StarRocks + Airflow
iceberg-polaris-spark-local-v0.1	pilot-stable *	Polaris RBAC/credential-vending catalog + Spark + StarRocks (fixed: 7-step Polaris 1.4.1 chain)
delta-hms-spark-trino-local-v0.1	pilot-stable *	Delta Lake on HMS + MySQL, read via Trino (fixed: --packages classpath + drop disabled register_table)

* = fixed during the 2026-07-17 VPS campaign to reach a working end-to-end install.

The compatibility lock schema (stacks/compatibility/<id>.lock.yaml) — one per stack: schema_version, stack_id, version_id, certified_at/certified_by/certified_on (host triple), status, status_notes (the human chronicle + honest evidence classification), **components[]** (image + **immutable tag** — floating tags like latest are forbidden — + registry/upstream URLs + notes), **constraints[]** (pairwise/N-way between + rule + verified proof string), host_requirements, **evidence[]** (append-only install records), and **incompatible[]** (known-bad combos with reason + workaround).

The certification ladder (maturity grades):

1. **candidate** — image tags verified to *exist*, but the *combination* has never been installed end-to-end. Empty evidence[].
2. **pilot-stable** — install completes and the lakehouse is readable (smoke may need a Skip on some OS).
3. **linux-stable** — verified end-to-end on a Linux VPS, smoke passes clean (no Skip).
4. **production** — additional hardening (TLS, backups, RBAC) certified.

The candidate ■ pilot-stable contract (the core invariant): status: candidate **iff** evidence[] is empty. Promotion is mechanical, not editorial — run the full pipeline end-to-end, append ≥1 passing evidence[] record, and in the **same commit** flip status → pilot-stable and bump version_id + certified_at. Evidence is **append-only** (a failed run is never edited away — a second record documents the fix), which is the project's "honest accounting" principle. A sibling <id>.upgrades.yaml offers a read-only "what could I bump?" surface that never applies anything.

components-catalog.yaml (the Knowledge Layer, ~2000 lines) feeds the Cart UI: goals[] (6 use-case cards), recommended_sets{} (named carts, each mirroring a stack's certified marriage), templates[] (8 use-case views with persona / anti-use-cases / compliance tags), categories[] (21 ordered component categories from table_format through security_hadoop, each component richly annotated with capabilities/best_for/why_this_version), and destinations[] (downstream BI tools).

version-compat-rules.yaml drives the **cascade picker** (selecting an anchor version filters dependents to compatible versions — a UI-scoping mechanism distinct from the lock's install-time enforcement).

compliance.yaml holds framing-only prose (hipaa/pci_dss/soc2/gdpr) where every entry must lead with a "reference framing, not certification" disclaimer.

The two heavyweight bundled stacks ship self-contained Docker contexts:

- **enterprise-hadoop-v1.0** (stacks/enterprise-hadoop/) — a single-host Docker replica of a real on-prem datalake (versions aligned byte-for-byte to a 2026-05-21 live scan): HDFS 3.4.1 (NameNode + 3 DataNodes), YARN 3.4.1 + MR History + Tez 0.10.4, Hive Metastore + HiveServer2 4.0.1, Hudi 1.0.1, Spark 3.4.4 (on YARN), StarRocks 4.0.0-rc01 FE+BE, Trino 480, PostgreSQL 14 + PgBouncer, Airflow

2.10.5 (Celery + RabbitMQ), and Prometheus + Grafana + Loki + Promtail. Apache Ranger 3.0.0 + Solr 8.11.4 are architecturally present (declared components, data volumes) but **not on the default up-path** — the Ranger plugin binaries must be built from source to enable enforcement. RAM min 20 / rec 48 GB — the heaviest stack.

- **streaming-local-v1.0** (stacks/streaming/) — a live CDC-into-Iceberg pipeline: Kafka 3.8.0 (KRaft, no Zookeeper) + Kafka UI, Flink 1.20.1 (custom `sl-flink:1.20-iceberg` image with Iceberg/Kafka/S3 connectors, JobManager + TaskManager), Iceberg REST 1.6.0, MinIO, Spark 3.5, and StarRocks 3.3.12. All services use `sl-` prefixes + remapped ports so it coexists with other stacks.

6.7 Day-2 operations (operator surface)

All reachable from the success screen and the CLI:

- **Smoke tests** — re-runnable end-to-end health checks.
- **Demo query + read-only SQL editor** — SELECT/SHOW/DESCRIBE/EXPLAIN/WITH only; 10 KB / 30 s limits; destructive statements rejected.
- **Service logs** — per-service `docker compose` log fetch + live stream.
- **Backups & restore** — metadata or full backups; create/list/restore; **scheduled backups** (enable/interval/kind) and non-destructive DR drills.
- **Lakehouse tables** — Iceberg namespace/table tree + schema/snapshot detail (30 s server cache).
- **Upload data (ingest)** — CSV drag-drop with schema preview → target DB/table; plus MySQL/Postgres sources. Universal ingest via Spark-Iceberg.
- **Security & TLS** — MinIO/StarRocks password rotation with a strength meter; self-signed cert generation (Let's Encrypt via the Caddy sidecar).
- **Certification evidence** — renders the stack lock file's `evidence[]`.
- **Export as GitOps bundle** — a tarball of `docker-compose.yml` + scrubbed `.env` + scripts + manifest.
- **Connect downstream tools (destinations)** — add BI destinations (Insyght / Tableau / Looker / Mode / Superset / Metabase / Power BI / custom JDBC) in three modes (**sql_pull** GA, **push_api** preview, **file_drop** stub); provisions a read-only StarRocks user; Fernet-encrypted credentials.
- **Day-2 controls** — status, re-run smoke, stop, clean (destroy volumes), uninstall & forget.
- **Ask Studio** — AI assistant grounded on the install's lock file/state/error catalog (needs `ANTHROPIC_API_KEY`).

6.8 AI-assisted provisioning & troubleshooting

Two AI surfaces, both driven via `litellm` (provider-agnostic) / the Anthropic SDK:

- **AI Build / provisioner** (`backend/ai_provisioner.py`, `ai_configurator.py`, `compat_ai.py`) — an "AI Build" path researches compatible versions, generates configs (Trino catalog files, post-start commands, connectivity checks), installs, and verifies. **Security note:** as of this session, all AI-emitted artifacts pass a trust boundary (`backend/ai_safety.py`) before execution (see §7).
- **Ask Studio assistant** — grounded troubleshooting over the install's lock file, live state, and a known-error catalog.

7. Security posture (current) & the hardening already done

Fixed this session — a confirmed remote-code-execution path. The AI provisioner asked an LLM for a JSON provisioning plan and executed it directly via `shell=True: post_start_commands`, `connectivity_checks`, and Trino catalog files all flowed `litellm.completion() → json.loads() → a shell`. A prompt injection smuggled through a stack name or component description could run arbitrary host commands.

The gate now in place (backend/ai_safety.py, 33 tests):

- `validate_catalog_filename()` — strict allowlist; blocks path traversal and shell-metacharacter filenames.
- `vet_provisioning_command()` — every command must lead with an allowlisted binary (docker control-plane verbs + read-only probes/filters) and contain no host-escape / exfiltration / container-breakout token (`--privileged`, `docker.sock`, reverse shells, pipe-to-shell, command substitution, `rm -rf /`, host bind-mounts, ...).
- Catalog **content** is now written over stdin (argv only) — never interpolated into a shell string (this also fixed a dead escaped variable that had left raw content interpolated).
- The same gate is applied to the custom-stack runner's AI-generated connectivity checks.

Threat model after the fix: even a fully compromised AI plan can, at worst, run read-only probes and docker control-plane ops against Studio-managed containers — no host escape, no exfiltration, no privileged containers.

Still open (tracked in §10): stacks run on the host Docker daemon with no sandbox/isolation policy; images are mostly tag-pinned, not digest-pinned; no image scanning; RBAC is opt-in bearer-token flags (with several WRITE-RISK routes reachable by VIEWER — a known gap in `docs/RBAC.md`), not SSO/OIDC; single-host only.

Existing controls: opt-in per-user RBAC (`LHS_RBAC_ENABLED`, sha256-hashed tokens, OWNER/ADMIN/OPERATOR/VIEWER); opt-in additive audit trail (`LHS_AUDIT_ENABLED`) with secret redaction and retention pruning; Fernet-encrypted destination credentials; a read-only SQL editor; and launcher warnings when bound to a public interface without a token.

8. Current state — verified facts (2026-07-17)

Item	State
Codebase	Synced to manaskiran/LakeHouse-Studio v0.6.2 (stable), <code>origin/main</code>
Test suite	498 passed / 0 failed (1 skipped, 1 xfail) — up from 444 passed / 24 failed at sync
Dependencies	Install cleanly (litellm needs a prebuilt wheel via <code>pip ≥ 25</code>)
Catalog integrity	Enforced by <code>scripts/catalog_lint.py</code> CI gate
AI-provisioning RCE	Gated (backend/ai_safety.py)

Certified stacks	ALL 12 pilot-stable — every installable stack was run end-to-end on the Finalert VPS on 2026-07-17 and carries a recorded evidence[] record. (hudi, delta, iceberg-polaris, fintech-compliance were fixed to get there; techsophy-sdp-hadoop-v1.0 is remote-cluster, health-check only.)
Production-readiness	■ not yet — pilot-stable = installs & works, <i>not</i> production-hardened. Demo creds, no SSO, host-Docker (no sandbox), tag-pinned/unscanned images, single-host. See §10 Phase 0–2.
Auth / RBAC	Opt-in bearer-token RBAC; not SSO/OIDC
Sandboxing	None — host Docker daemon
Supply chain	Mostly tag-pinned; no image scanning
HA / DR	Backup + DR drills present; single-host only

9. Council assessment — scored review

A five-model council (Claude Opus, Claude Sonnet, Claude Fable, the Google/agy lane, and OpenAI Codex) independently scored the product. Category scores are the mean of the five panellists.

Dimension	Score (avg)	Notes
Innovation / differentiation	8.1 / 10	Evidence-backed compatibility-as-a-contract is a real, defensible moat
UX / product design	7.5 / 10	"Shop → pre-flight → live-streamed install → day-2" is a genuinely good operator experience
Testing & QA	7.4 / 10	498/0 green + clean installs; coverage is thin on the uncertified stacks
Architecture & code quality	7.0 / 10	Clean FastAPI/SPA/CLI split + lock-file discipline; host-Docker coupling caps the ceiling
Functionality & completeness	6.4 / 10 → <i>revised upward</i>	Scored when only 4/12 stacks were proven; all 12 are now pilot-stable with VPS evidence , so this dimension is materially stronger than the panel could credit
Documentation	5.9 / 10	Strong compatibility/certification artifacts; per-stack evidence & runbooks lag

Security posture	4.2 / 10	RCE caught and gated, but no sandbox, tag-not-digest pins, no image scan, no SSO
Enterprise readiness	3.4 / 10	Single-host, no SSO/OIDC, no isolation — pilot-grade, not production

Individual overall scores: Opus 6.0 · Sonnet 5.4 · Fable 6.4 · agy (Google) 5.5 · Codex 7.2.

■ ■ Overall score: 6.1 / 10

Council consensus (verbatim themes): *"An impressively conceived, well-tested product with a differentiated compatibility-contract model and excellent install UX, but honestly self-assessed as early."*

The ceiling is not vision or engineering hygiene — it is **breadth of evidence and hardening**. Every panellist converged on the same lever: **get 5–6 stacks pilot-stable, digest-pin and scan images, add OIDC + a runtime sandbox, and the score jumps ~1.5 points.**

Note — the evidence gap the panel flagged is now closed. The council scored the product when only ~4 of 12 stacks were proven, and every panellist named *breadth of install evidence* as the primary limiter. On 2026-07-17, **all 12 stacks were installed end-to-end on the Finalert VPS and promoted to pilot-stable with recorded evidence** (the 4 broken ones — hudi/delta/polaris/fintech — were root-caused and fixed). That directly resolves the functionality/evidence lever the panel pointed at, so the effective score is now higher than 6.1 on that axis. The score's remaining ceiling is the **security & enterprise-readiness** work (§10 Phase 0–2), which is unchanged.

10. The road to enterprise grade — step by step

The council converged on one lever: the ceiling is **breadth of evidence and hardening**, not vision. This section is the detailed, sequenced plan. Each step has a concrete definition-of-done. Phases are ordered by "expensive if wrong" first.

Phase 0 — Close the safety boundary (*target: ~2 weeks; blocks any non-trusted use*)

#	Step	Concrete tasks	Done when
P0.1	Validate composed stacks	Add a schema gate in <code>stack_composer / image_builder</code> that rejects <code>privileged</code> , <code>pid: host</code> , <code>network_mode: host</code> , host bind-mounts (<code>/var/run/docker.sock</code> , <code>/</code> , <code>/etc</code>), and <code>cap_add</code> outside an allowlist. Reuse the token list already in <code>backend/ai_safety.py</code> .	A malicious component spec is refused with a test proving it

P0.2	Container runtime policy	Run generated stacks with <code>--cap-drop=ALL</code> (minimal re-adds), read-only root FS where feasible, CPU/mem/pids limits, an isolated bridge network with no host route. Evaluate rootless Docker / a socket proxy.	Stacks come up under the hardened profile; smoke still passes
P0.3	Human-in-the-loop for AI plans	Show a diff/preview of the resolved compose + AI commands and require explicit confirmation before launch. Default AI provisioning off unless an API key + opt-in are present.	No AI-composed stack launches without a confirm step
P0.4	Secrets hygiene	Source all creds (litellm/Anthropic keys, DB passwords) from env/secret-manager; never log; force non-default credential generation for Ranger/Hive/StarRocks/Airflow at provision time; add a <code>gitleaks/ecc-agentshield</code> CI gate.	Secret-scan CI is green; no secret appears in logs/generated configs
P0.5	Egress control	Restrict outbound network on built/run images to prevent exfiltration from a compromised stack.	Documented egress policy applied to the run profile

Phase 1 — Prove the stacks (*target: 2-6 weeks*)

#	Step	Concrete tasks	Done when
---	------	----------------	-----------

P1.1	Earn install evidence	Run the full pipeline end-to-end for each candidate stack (install→health→teardown) via <code>scripts/install_harness.py</code> ; append <code>evidence[]</code> ; flip <code>status → pilot-stable</code> in the same commit. Prioritize enterprise-hadoop-v1.0 and streaming-local-v1.0 (the headline new stacks).	≥5-6 stacks pilot-stable with recorded evidence
P1.2	Release gates in CI	Block release unless: unit suite green, <code>catalog_lint</code> clean, compose-policy validation passes, security scan passes, and ≥1 clean install/health run per promoted stack.	A red gate blocks a release in CI
P1.3	Digest-pin images	Convert lock files from <code>image:tag</code> to <code>image@sha256:...</code> ; pin <code>bitsondatadev/hive-metastore:latest</code> (a known floating tag) first.	No floating/tag-only pins remain in locks
P1.4	Supply-chain scanning	Trivy/Grype on every image before it enters a stack; <code>pip-audit</code> on Python deps; generate an SBOM per release.	Scans run in CI; criticals block

Phase 2 — Enterprise access & governance (target: 1-2 months)

#	Step	Concrete tasks	Done when
P2.1	AuthN	SSO/OIDC login; httpOnly session cookies; CSRF protection on state-changing routes; auth middleware on all API endpoints (audit the current gaps where VIEWER can reach WRITE-RISK routes).	OIDC login works; no unauthenticated write route

P2.2	AuthZ	Promote RBAC from flags to enforced per-resource policies; wire Ranger/Polaris policy stores for data-plane authz; make the audit log tamper-evident (append-only, hash-chained).	RBAC denies a VIEWER a write route in a test; audit chain verifies
P2.3	Multi-tenancy	Formalize per-tenant network/volume/project-name isolation (partly present via install-specific LHS_NET + named volumes); wire the v1 multi-tenant SQLite schema.	Two tenants' installs are provably isolated
P2.4	Data governance	Promote OpenLineage lineage + data-quality checks out of candidate; document retention/PII handling for regulated templates.	Lineage installs; DQ checks run on a certified stack

Phase 3 — Reliability, observability, scale (*ongoing*)

- **P3.1 Observability GA** — promote Prometheus + Grafana + Loki from candidate to a real install path; structured JSON logging; alerting rules; per-stack dashboards.
- **P3.2 HA / multi-host** — a documented + tested multi-node path; resource quotas; graceful degradation.
- **P3.3 DR** — extend backup/DR drills to full restore-from-scratch rehearsals with recorded RTO/RPO.
- **P3.4 Upgrade/migration paths** — every lock change ships a migration note; test in-place upgrades between certified versions.

Phase 4 — Quality & release engineering (*ongoing*)

- **P4.1 Coverage to 80%+** (unit + integration + E2E); add integration tests that actually spin up compose fragments.
- **P4.2 Overlay-drift audit** — because the v0.6.2 sync was a manual overlay onto an unrelated history, add a periodic diff against upstream to catch silent semantic drops.
- **P4.3 Accessibility** — keep the VPAT current; add automated a11y checks (fix the known 2.2.2 animation-pause and 2.4.1 skip-link/landmark gaps).

Definition of "enterprise grade" (exit criteria)

1. No untrusted input (user **or** LLM) can reach a host shell, the Docker daemon, or an unvalidated compose — enforced by tests.
2. Every advertised stack is `pilot-stable` (ideally `linux-stable`) with recorded install evidence and digest-pinned, scanned images.
3. SSO + enforced RBAC + tamper-evident audit on every endpoint and data plane.
4. Multi-tenant isolation, an HA path, and rehearsed DR with published RTO/RPO.
5. A green CI release gate: `unit + catalog_lint + compose-policy + security scan + per-stack smoke evidence`.

11. Appendix — dependencies, launch, glossary

Launch: `bash run.sh` (macOS/Linux/Git Bash) or `powershell -ExecutionPolicy Bypass -File .\run.ps1` (Windows). Both create `.venv`, `pip install -r requirements.txt`, and start uvicorn on `LHS_HOST:LHS_PORT` (default `127.0.0.1:7878`).

Runtime deps (15 pins): `fastapi 0.115.5`, `uvicorn[standard] 0.32.1`, `pydantic 2.10.3`, `python-multipart`, `jinja2`, `pyyaml`, `psutil` (host inspection), `cryptography` (Fernet + TLS), `psycopg[binary]` + `pymysql` (DB drivers), `anthropic` + `litellm` + `python-dotenv` (AI), `click` + `rich` (CLI), `typing_extensions`. Requires Python 3.11+, Docker, bash + git.

Feature flags (env): `LHS_RBAC_ENABLED`, `LHS_AUDIT_ENABLED`, `LHS_AUDIT_SCHEDULER_ENABLED`, `LHS_*_ENABLED` (Airflow/Dagster/Superset/observability overlays), `LHS_AUTH_TOKEN`, `LHS_HOST/LHS_BIND/LHS_PORT`, `ANTHROPIC_API_KEY` / `LITELLM_*`.

Glossary: *Lock file* — per-stack `stacks/compatibility/<id>.lock.yaml` pinning exact image tags + constraints + evidence. *Candidate / pilot-stable* — certification states; `pilot-stable` requires ≥ 1 recorded passing install. *Recommended set* — the cart-facing component marriage for a stack. *Fragment* — an additive `docker-compose.fragment.yaml` supplying services the base compose omits. *Override compose* — additive sidecar files (TLS/monitoring/JDBC) the operator runs themselves.